



Pandas

- What is it?
- Why Pandas?
- Structures data
- Series
- Series and operations mathematics
- Series and masks
- Missing
- Statistics
- Dataframes
- Data from csv
- Indexing
- Merging
- Grouping
- Cheatsheet

Pandas: What is it?

- **Pandas** is a Python library that provides high-performance data structures .
- The name **Pandas** comes from the fusion of the words *Panel* and *Data* .
- **Pandas** is easy to use alongside other Python analysis tools.
- **Pandas** depends on **NumPy** , which means it is built " *on top* " of the **NumPy** library and *depends on it* .
- **Pandas** only works if you also have **NumPy** installed :
 - If **NumPy** is already installed, **Pandas** is installed as usual
 - Otherwise, installing **Pandas** will also install its " *dependencies* ", including **NumPy** .
- It is the basic tool, together with **NumPy** , for representing scientific information and datasets which will then be given as input to AI and ML tools.
- It is *open-source* and **freely accessible** (BSD, free license).

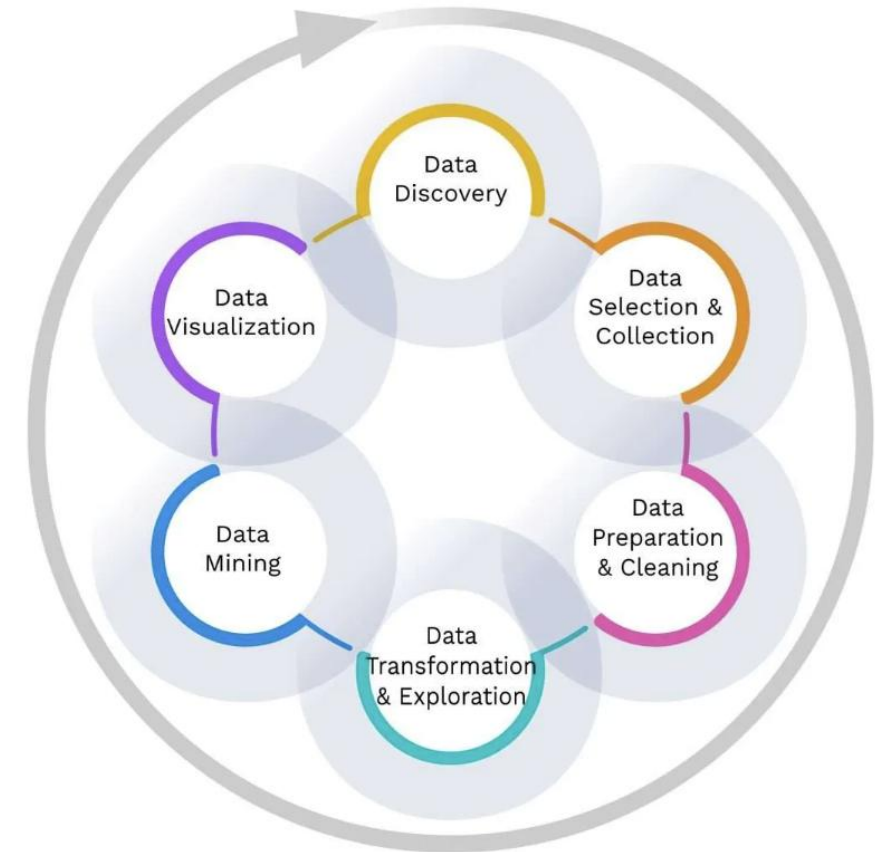


More docs at:

<https://pandas.pydata.org/docs/>

Pandas: Why ?

- Using **Pandas** we can perform the 5 (+ 1) typical steps of data processing and analysis:
- Some typical possible operations offered by the tool are:
 - Management of missing data
 - Dividing, indexing and searching information in large data sets
 - Ability to modify the data structure (deletion, addition, manipulation of rows and columns)
 - Data grouping and transformation
 - Time series management.



Le 6 (+1) fasi della Data Science

Pandas: Data Structures

- Like **NumPy** , it offers functionality of saving and loading data from Excel, csv, databases...
- The data structures generally managed by **Pandas** are tabulated (i.e. in the form of tables) and sequential:
 - **Series** : data in one-dimensional format (such as *arrays* or *lists*), for managing sequences or time series; can contain objects of arbitrary type.
 - **Dataframe** : data in two-dimensional format (*tables* or *matrices*).
- NOTE: Each column of a Dataframe is a Series.

Pandas: Series

- The Series associates with each position an index inside them .
- The index is not necessarily numeric .
- As with arrays and lists , the index can to be used to log in to elements contents in the **Series** .
- It's possible specify a **Series** *without an index* .
- `import pandas as pd`

```
my_series = pd.Series(["Mercury" , "Venus", "Earth"])
print(my_series) #Out:    0 Mercury
                        1 Venus
                        2 Earth
                        dtype : object
```

- <https://pandas.pydata.org/docs/reference/api/pandas.Series.html>

Pandas: Series Declaration

- It's possible specify a **Series** *with index explicit*.
 - `my_series = pd.Series(["Mercury", "Venus", "Earth"], index=[1, 2, 3])`
`print(my_series)` #Out: 1 Mercury
2 Venus
3 Earth
dtype : object
- It's possible specify a **Series** *via a dictionary*.
 - `my_series = pd.Series({"1": "Mercury", "2": "Venus", "3": "Earth"})`
`print(my_series)` #Out: 1 Mercury
2 Venus
3 Earth
dtype : object

Pandas: Series Declaration

- It's possible specify a **Series** *through lists* .

It's possible log in through index *numeric* or *symbolic* .

- ```
planets = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn",
 "Uranus", "Neptune"]
```

```
diameters = [5, 12, 13, 7, 143, 121, 51, 50]
```

```
my_series = pd.Series(diameters, index=planets)
```

```
print(my_series["Mars"])
```

```
print(my_series[3])
```

- By accessing an item which is not on the list , comes returned a **KeyError**.

- ```
print(my_series["Pluto"])          #Out: KeyError
```

```
print(my_series.get["Pluto"])      #Out: None
```

Pandas: Series and Mathematical Operations

- If we apply a mathematical calculation *directly* to one **Series**, the calculation is *applied to each element* of the Series itself .

```
• print(my_series * 1000) #Out: Mercury 5000
                             Venus 12000
                             Earth 13000
                             Mars 7000
                             Jupiter 143000
                             Saturn 121000
                             Uranus 51000
                             Neptune 50000
                                dtype : int64
```

Pandas: Series and Masks

- It's possible to apply to **Series** *type operators boolean* . The result is a **Mask** :

- `print( my_series < 10)` #Out:

Mercury	True
Venus	False
Earth	False
Mars	True
Jupiter	False
Saturn	False
Uranus	False
Neptune	False

dtype : bool

Pandas: Series and Masks

- Masks can be used for filtering the elements of one Series , given a defined condition:

- ```
print(my_series[my_series < 10])
```

 #Out: Mercury 5  
Mars 7  
dtype : int64

- The concept of **Mask** can to be matched to the assignment of information inside Python variables:

- ```
mask = my_series < 10  
print(mask)  
filtered_series = my_series[mask]  
print(filtered_series)
```

 #Out: Mercury 5
Mars 7
dtype : int64

Pandas: Series and Operations

- It is possible to carry out operations also *between two Series* . In this case, you need to pay attention to the missing data:

- ```
n1 = ["Alice", "Bob", "Charlie", "David"]
```

```
v1 = [7, 9, 3, 10]
```

```
n2 = ["Bob", "Charlie", "David", "Eric"]
```

```
v2 = [11, 5, 1, 8]
```

```
series1 = pd.Series(v1, index=n1)
```

```
series2 = pd.Series(v2, index=n2)
```

```
print(series1 + series2)
```

```
#Out: Alice NaN
```

```
 Bob 20.0
```

```
 Charlie 8.0
```

```
 David 11.0
```

```
 Eric NaN
```

```
dtype : float64
```

- The resulting series has as its index the union of the two groups of indices and as its values the sum of the values with the same index.
- Why is the return value **NaN for some keys** ?

# Pandas: Missing Data

---

- **NaN** (i.e., "*not a number* ") is one symbolic constant meaning that:
  - the value it represents is *numeric*;
  - the value it represents is *invalid* or *not defined*.
- NOTE: **NaN** spreads very easily if the dataset "is not **clean**" before his usage.
- ```
series3 = series 1 + series2  
print(series3["Alice"] + series1["David"]) #Out: NaN
```

Pandas: Missing Data

- Handling of missing data is a fundamental step in the preparation phase of the dataset.
- Handling methods vary from scenario to scenario.

- **print**(s3.dropna())
#Out: Bob 20.0
Charlie 8.0
David 11.0
dtype: float64

- **print**(s3.fillna(0))
#Out: Alice 0.0
Bob 20.0
Charlie 8.0
David 11.0
Eric 0.0
dtype: float64

- <https://pandas.pydata.org/docs/reference/api/pandas.Series.dropna.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.Series.fillna.html>

Pandas: Missing Data

- It's possible use specific methods (e.g.: the add method instead of the "+" operator) to handle the missing data:

- `series4 = series1.add(series2, fill_value = 0)`

```
print(series4) # Out:      Alice 7.0
                Bob 20.0
                Charlie 8.0
                David 11.0
                Eric 8.0
                dtype : float64
```

- <https://pandas.pydata.org/docs/reference/api/pandas.Series.add.html>

Pandas: Statistics

- Exist methods integrated into **Pandas** for extracting statistics.
- `print(s.sum())`
`print(s.prod())`
`print(s.max())`
`print(s.argmax())`
`print(s.mean())`
`print(s.var())`
`print(s.std())`
`print(s.median())`
`print(s.describe())`
- <https://pandas.pydata.org/docs/reference/api/pandas.Series.add.html>

Pandas: Dataframes

- It is the two-dimensional analogue from the **Series** .
- It is an information table that can contain also *heterogeneous data*.
- A **Dataframe** has three fundamental characteristics:
 1. The **index** , which contains labels for the rows;
 2. The **labels**, for the columns;
 3. The **shape** , describing the size of the table .
- A *single-column* of a **Dataframe** is a **Series** .
- <https://pandas.pydata.org/docs/reference/frame.html>

Pandas: Dataframes Declaration

- It is possible to initialize a **Dataframe** starting from a *Series dictionary* :
 - **Keys** become *the names of the columns* , while **indexes of the Series** become *indexes of the lines* .
 - ```
d={"name": pd.Series(["Harry", "Ronald", "Hermione", "Albus"]),
 "surname": pd.Series(["Potter", "Weasley", "Granger", "Dumbledore"])}
df = pd.DataFrame(d)
print(df)
```

```
Out: name surname
 0 Harry Potter
 1 Ronald Weasley
 2 Hermione Granger
 3 Albus Dumbledore
```

    - ```
print(df.index)
```

```
# Out: RangeIndex(start=0, stop=4, step=1)
```
 - ```
print(df.shape)
```

```
Out: (4, 2)
```
  - ```
print(df.columns)
```

```
# Out: Index(['name', 'surname'], dtype = 'object')
```

Pandas: Dataframes Declaration

- As for the Series, if during the creation of a Dataframe starting from existing data structures, there is no correspondence between indices or there are missing data, in their place is put the symbolic value **NaN**.
- ```
d = {"x": pd.Series([0, 0], index=["a", "b"]), "y": pd.Series([0, 0], index=["b", "c"])}
df = pd.DataFrame(d)
print(df)
```

# Out:

|   | x   | y   |
|---|-----|-----|
| a | 0.0 | NaN |
| b | 0.0 | 0.0 |
| c | NaN | 0.0 |
- In this case, not having in the series x the value "c", to its place was inserted **NaN**; the same mechanism was followed for y, at index "a".

# Pandas: Dataframes Declaration

- It's possible initialize a **Dataframe** with *a list dictionary*:
  - Even in this case, the dictionary **keys** become the *names of the columns* , while **row** numbers become *indexes of the lists* (i.e., their *positions*).
  - ```
d = {"name": ["Harry", "Ronald", "Hermione", "Albus"], "surname": ["Potter", "Weasley", "Granger", "Dumbledore"]}
```

```
df=pd.DataFrame(d)
```

```
print(df)
```

# Out:	name	surname
0	Harry	Potter
1	Ronald	Weasley
2	Hermione	Granger
3	Albus	Dumbledore

Pandas: Dataframes Declaration

- In **Series**, as well as in the creation of the **Dataframe** is possible specify an *index* :

```
d={ "name": [ "Harry" , "Ronald", "Hermione",  
            "Albus"], "surname": [ "Potter", "Weasley",  
            "Granger", "Dumbledore"] }  
df=pd.DataFrame(d, index=["a", "b", "c", "d"] )  
print(df)
```

```
# Out:      name      surname  
a      Harry      Potter  
b      Ronald      Weasley  
c      Hermione    Granger  
d      Albus       Dumbledore
```

Pandas: Data from .csv

- If not explicitly specified, indexes of the lines of the dataframe are created automatically from 0 to N-1, with a total number of N lines.

- ```
df = pd.read_csv("planets.csv")
```

```
print(df)
```

```
Out:
```

|   | Planet  | Diameter | Gravity | DayLength | MeanTemperature |
|---|---------|----------|---------|-----------|-----------------|
| 0 | Mercury | 4879     | 3.7     | 4222.6    | 167             |
| 1 | Venus   | 12104    | 8.9     | 2802.0    | 464             |
| 2 | Earth   | 12756    | 9.8     | 24.0      | 15              |
| 3 | Mars    | 6792     | 3.7     | 24.7      | -65             |
| 4 | Jupiter | 142984   | 23.1    | 9.9       | -110            |
| 5 | Saturn  | 120536   | 9.0     | 10.7      | -140            |
| 6 | Uranus  | 51118    | 8.7     | 17.2      | -195            |
| 7 | Neptune | 49528    | 11.0    | 16.1      | -200            |

# Pandas: Data from .csv

- Alternatively, it is possible to *use a **column** as index*.

- ```
df = pd.read_csv("planets.csv", index_col="Planet")
```

```
print(df)
```

```
# Out:
```

Planet	Diameter	Gravity	DayLength	MeanTemperature
Mercury	4879	3.7	4222.6	167
Venus	12104	8.9	2802.0	464
Earth	12756	9.8	24.0	15
Mars	6792	3.7	24.7	-65
Jupiter	142984	23.1	9.9	-110
Saturn	120536	9.0	10.7	-140
Uranus	51118	8.7	17.2	-195
Neptune	49528	11.0	16.1	-200

- `print(df.columns)` what does it return?
- `print(df.index)` what does it return?
- `print(df.shape)` what does it return?

Pandas: Indexing

- # Select a column
`print(df["Gravity"])`
- # Select a set of columns
`print(df[["Gravity", "MeanTemperature"]])`
- # Select a line if the index is symbolic
`print(df.loc["Venus"])`
- # Select a portion of lines with *slicing*
`print(df[2:5])`
- # Select lines based on a vector boolean
`print(df[df["Gravity"]>10])`
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

Pandas: Indexing

- ```
df = pd.DataFrame([[1, 2], [4, 5], [7, 8]], index=['cobra', 'viper', 'sidewinder'], columns=['max_speed', 'shield'])
```

```
print(df)
```

```
Out: max_speed shield
```

|            |   |   |
|------------|---|---|
| cobra      | 1 | 2 |
| viper      | 4 | 5 |
| sidewinder | 7 | 8 |
- ```
df.loc['viper']
```

```
# Out:      max_speed  4
```

shield	5
--------	---

```
Name: viper,
```

```
dtype : int64
```
- ```
df.loc[['viper', 'sidewinder']]
```

```
Out: max_speed shield
```

|            |   |   |
|------------|---|---|
| viper      | 4 | 5 |
| sidewinder | 7 | 8 |
- ```
df.loc['cobra', 'shield']
```

```
# Out: 2
```
- ```
df.loc['cobra':'viper', 'max_speed']
```

```
Out: cobra 1
```

|       |   |
|-------|---|
| viper | 4 |
|-------|---|

```
Name: max_speed,
```

```
dtype: int64
```

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html>

# Pandas: Indexing

- # Select a row/col given the label  
`print(df.loc[0])`
- Some possible inputs are :
  - A single label , e.g., '5' or 'a' (note: '5' in this case is interpreted as a *label* of the index, or as if it is a **string** and not as an integer positional index).
  - A list or array of labels, e.g., ['a', 'b', 'c'] .
  - A label slice, e.g., 'a' : 'f' (note: in **Pandas** , unlike Python, **slices** return the interval **from the first to the last value included** , and not like in Python where *the last value is excluded* ). Can you guess why?

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html>

# Pandas: Indexing

- # Select a row/col, given the *numeric position*, from 0 to length-1 of the selected axis  
`print(df.iloc[0])`
- Some possible inputs are:
  - An integer, e.g., 5;
  - A list or array of integers, e.g., [4, 3, 0];
  - A slice of integers, e.g., 1:7;
  - An array of booleans.
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

# Pandas: Indexing

- ```
mydict = [{'a': 1, 'b': 2, 'c': 3, 'd': 4},  
{ 'a': 100, 'b': 200, 'c': 300, 'd ': 400}, { 'a': 1000, 'b': 2000, 'c': 3000, 'd': 4000}]  
df = pd.DataFrame(mydict)  
print(df)
```

# Out:	a	b	c	d
0	1	2	3	4
1	100	200	300	400
2	1000	2000	3000	4000
- ```
print(df.iloc[0])
```

```
Out: a 1
 b 2
 c 3
 d 4
 Name: 0,
 dtype: int64
```

**(note: this is a series!)**

- ```
print(df.iloc[[0]])
```

```
# Out:      a      b      c      d  
        1      2      3      4
```

(note: it's a dataframe!)
- ```
print(df.iloc[[0, 1]])
```

```
Out: a b c d
 0 1 2 3 4
 1 100 200 300 400
```
- ```
print(df.iloc[:3])
```

```
# Out:      a      b      c      d  
      0      1      2      3      4  
      1     100     200     300     400  
      2    1000    2000    3000    4000
```
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

Pandas: Indexing

- ```
mydict = [{'a':1, 'b':2, 'c':3, 'd':4},
{ 'a':100, 'b':200, 'c':300, 'd':400}, { 'a':1000, 'b':2000, 'c':3000, 'd':4000}]
df = pd.DataFrame(mydict)
print(df)
```

| # Out: | a    | b    | c    | d    |
|--------|------|------|------|------|
| 0      | 1    | 2    | 3    | 4    |
| 1      | 100  | 200  | 300  | 400  |
| 2      | 1000 | 2000 | 3000 | 4000 |

- ```
print(df.iloc[0, 1])  
# Out: 2
```

- ```
print(df.iloc[[0, 2], [1,3]])
Out:
```

|   | b    | d    |
|---|------|------|
| 0 | 2    | 4    |
| 2 | 2000 | 4000 |

- ```
print(df.iloc [1:3, 0:3])  
# Out:
```

	a	b	c
1	100	200	300
2	1000	2000	3000

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

Pandas: Merging

- It is possible combine two dataframes with one merging operation .
- Merging is based on shared columns between two tables .
- ```
planets_1 = pd.DataFrame({"Planet": ["Earth" , "Mars" , "Jupiter"],
 "Gravity": [9.8, 3.7, 23.1]})
print(planets_1)
```

| # Out: | Planet  | Gravity |
|--------|---------|---------|
| 0      | Earth   | 9.8     |
| 1      | Mars    | 3.7     |
| 2      | Jupiter | 23.1    |
- ```
planets_2 = pd.DataFrame({"Planet": ["Mars", "Jupiter", "Neptune"], "  
    MeanTemperature" : [- 65, -110, -200 ]})  
print(planets_2)
```

# Out:	Planet	MeanTemperature
0	Mars	-65
1	Jupiter	-110
2	Neptune	-200

Pandas: Merging

- `print(pd.merge(planets_1, planets_2, on="Planet", how="inner"))`

```
# Out:      Planet      Gravity  MeanTemperature
      0      Mars        3.7             -65
      1    Jupiter       23.1            -110
```

- If two dataframes have a column with the same name , **Pandas**, *via merging* :
 - reports the common column as the common access key for each line;
 - reports all the others columns of the two dataframes;
 - reports for each line from the column in common only the keys (i.e. , perform a intersection between the values)
 - populates all the others columns with values relative to the first or second dataframe (similar to a join in a database).

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

Pandas: Merging

- `print(pd.merge(planets_1, planets_2, on="Planet", how="inner"))`

```
# Out:
      Planet  Gravity  MeanTemperature
0    Mars      3.7      -65
1  Jupiter     23.1     -110
```

- Parentheses about "**how**":
 - inner discards all the rows which are not in common ;

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

Pandas: Merging

- `print(pd.merge(planets_1, planets_2, on="Planet", how="left"))`

```
# Out:
      Planet  Gravity  MeanTemperature
0    Earth      9.8         NaN
1    Mars      3.7        -65
2  Jupiter     23.1       -110
```

- Parentheses about "**how**":
 - **left** *discards all uncommon rows in the right dataframe*; uncommon values in the left dataframe, instead, **are maintained** ; possible missing data are filled by the placeholder **NaN** ;

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

Pandas: Merging

- `print(pd.merge(planets_1, planets_2, on="Planet", how="right"))`

```
# Out:
      Planet  Gravity  MeanTemperature
0      Mars    3.7      -65
1    Jupiter   23.1     -110
2    Neptune   NaN     -200
```

- Parentheses about "**how**":
 - **right** *discards all uncommon rows in the left dataframe*; uncommon values in the right dataframes, instead, **are maintained** ; eventual missing data are filled by the *placeholder NaN*;
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

Pandas: Merging

- `print(pd.merge(planets_1, planets_2, on="Planet", how="outer"))`

```
# Out:
      Planet  Gravity  MeanTemperature
0    Earth      9.8          NaN
1    Mars      3.7          -65
2  Jupiter     23.1         -110
3  Neptune     NaN         -200
```

- Parentheses about "**how**":
 - **outer** keeps all rows for each common and uncommon value (instead of the **intersection**, performs the **union** of the two dataframes) .

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

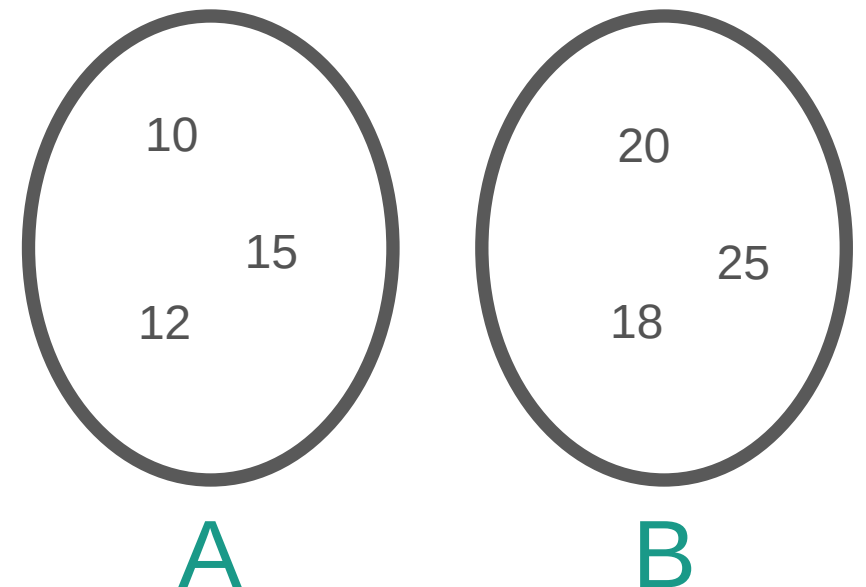
Pandas: Grouping

- In Pandas, "**grouping**" refers to the process of *dividing a data frame into groups based on the values of one or more columns*.
- This operation is often followed by the application of an aggregated function to each group .

```
data = {  
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],  
    'Value': [10, 20, 15, 25, 12, 18]  
}
```

```
df = pd.DataFrame(data)
```

```
# Group for column 'Category'  
groups = df.groupby('Category')  
cat_sum = groups['Value'].sum()  
print(cat_sum)  
# Out: A 37, B 63
```



Pandas: Grouping

- Grouping in Pandas can be executed on more columns at the same time.

- ```
data = {
 'Category': ['A', 'B', 'A', 'B', 'A', 'B'],
 'Value': [10, 20, 15, 25, 12, 18] ,
 'More': ['a', 'b', 'a', 'b', 'a', 'b']
}
```

```
df = pd.DataFrame(data)
```

```
Group for columns 'Category' and 'More'
multiple_groups = df.groupby(['Category', 'More'])
```

```
Applying an aggregated function to groups
sum_by_category_other = multiple_groups['Value'].sum()
print(sum_by_category)
Out: A a 37, B b 63
```

# Pandas: Cheatsheet

[Links](#)

## Python For Data Science Cheat Sheet

### Pandas

Learn Python for Data Science Interactively at [www.DataCamp.com](https://www.datacamp.com)



## Reshaping Data

### Pivot

```
>>> df3 = df2.pivot(index='Date',
 columns='Type',
 values='Value')
```

Spread rows into columns

|   | Date       | Type | Value  |
|---|------------|------|--------|
| 0 | 2016-03-01 | a    | 11.432 |
| 1 | 2016-03-02 | b    | 13.031 |
| 2 | 2016-03-01 | c    | 20.784 |
| 3 | 2016-03-03 | a    | 99.906 |
| 4 | 2016-03-02 | a    | 1.303  |
| 5 | 2016-03-03 | c    | 20.784 |

|   | Type       | a      | b      | c      |
|---|------------|--------|--------|--------|
| 0 | 2016-03-01 | 11.432 | NaN    | 20.784 |
| 1 | 2016-03-02 | 1.303  | 13.031 | NaN    |
| 2 | 2016-03-03 | 99.906 | NaN    | 20.784 |

### Pivot Table

```
>>> df4 = pd.pivot_table(df2,
 values='Value',
 index='Date',
 columns='Type')
```

Spread rows into columns

### Stack / Unstack

```
>>> stacked = df5.stack()
>>> stacked.unstack()
```

Pivot a level of column labels  
Pivot a level of index labels

|   |   | 0        | 1        |
|---|---|----------|----------|
| 1 | 5 | 0.233482 | 0.390959 |
| 2 | 4 | 0.184713 | 0.237102 |
| 3 | 3 | 0.433522 | 0.429401 |

Unstacked

|   | 5        | 4        | 3        |
|---|----------|----------|----------|
| 1 | 0.233482 | 0.390959 | 0.184713 |
| 2 | 0.184713 | 0.237102 | 0.433522 |
| 3 | 0.433522 | 0.429401 | 0.237102 |

Stacked

### Melt

```
>>> pd.melt(df2,
 id_vars=["Date"],
 value_vars=["Type", "Value"],
 value_name="Observations")
```

Gather columns into rows

|   | Date       | Type | Value  |
|---|------------|------|--------|
| 0 | 2016-03-01 | a    | 11.432 |
| 1 | 2016-03-02 | b    | 13.031 |
| 2 | 2016-03-01 | c    | 20.784 |
| 3 | 2016-03-03 | a    | 99.906 |
| 4 | 2016-03-02 | a    | 1.303  |
| 5 | 2016-03-03 | c    | 20.784 |

|    | Date       | Observations |
|----|------------|--------------|
| 0  | 2016-03-01 | a            |
| 1  | 2016-03-02 | b            |
| 2  | 2016-03-01 | c            |
| 3  | 2016-03-03 | a            |
| 4  | 2016-03-02 | a            |
| 5  | 2016-03-03 | c            |
| 6  | 2016-03-01 | Value        |
| 7  | 2016-03-02 | Value        |
| 8  | 2016-03-01 | Value        |
| 9  | 2016-03-03 | Value        |
| 10 | 2016-03-02 | Value        |
| 11 | 2016-03-03 | Value        |

## Iteration

```
>>> df.iteritems()
>>> df.iterrows()
```

(Column-index, Series) pairs  
(Row-index, Series) pairs

## Advanced Indexing

Also see NumPy Arrays

**Selecting**

```
>>> df3.loc[:, (df3>1).any()]
>>> df3.loc[:, (df3>1).all()]
>>> df3.loc[:, df3.isnull().any()]
>>> df3.loc[:, df3.notnull().all()]
```

**Indexing With isin**

```
>>> df[(df.Country.isin(df2.Type))]
>>> df3.filter(items=["a", "b"])
>>> df.select(lambda x: not x%5)
```

**Where**

```
>>> s.where(s > 0)
```

**Query**

```
>>> df6.query('second > first')
```

Select cols with any vals > 1  
Select cols with vals > 1  
Select cols with NaN  
Select cols without NaN  
Find same elements  
Filter on values  
Select specific elements  
Subset the data  
Query DataFrame

## Setting/Resetting Index

```
>>> df.set_index('Country')
>>> df4 = df.reset_index()
>>> df = df.rename(index=str,
 columns={"Country": "entry",
 "Capital": "cptl",
 "Population": "ppltn"})
```

Set the index  
Reset the index  
Rename DataFrame

## Reindexing

```
>>> s2 = s.reindex(['a', 'c', 'd', 'e', 'b'])
```

### Forward Filling

```
>>> df.reindex(range(4),
 method='ffill')
```

| Country | Capital | Population |
|---------|---------|------------|
| 0       | Belgium | Brussels   |
| 1       | India   | New Delhi  |
| 2       | Brazil  | Brasilia   |
| 3       | Brazil  | Brasilia   |

### Backward Filling

```
>>> s3 = s.reindex(range(5),
 method='bfill')
```

|   | 0 | 3 |
|---|---|---|
| 0 | 1 | 3 |
| 1 | 2 | 3 |
| 2 | 3 | 3 |
| 3 | 4 | 3 |

## Multindexing

```
>>> arrays = [np.array([1,2,3]),
 np.array([5,4,3])]
>>> df5 = pd.DataFrame(np.random.rand(3, 2), index=arrays)
>>> tuples = list(zip(*arrays))
>>> index = pd.MultiIndex.from_tuples(tuples,
 names=['first', 'second'])
>>> df6 = pd.DataFrame(np.random.rand(3, 2), index=index)
>>> df2.set_index(["Date", "Type"])
```

## Duplicate Data

```
>>> s3.unique()
>>> df2.duplicated('Type')
>>> df2.drop_duplicates('Type', keep='last')
>>> df.index.duplicated()
```

Return unique values  
Check duplicates  
Drop duplicates  
Check index duplicates

## Grouping Data

### Aggregation

```
>>> df2.groupby(by=["Date", "Type"]).mean()
>>> df4.groupby(level=0).sum()
>>> df4.groupby(level=0).agg({'a': lambda x: sum(x)/len(x),
 'b': np.sum})
```

### Transformation

```
>>> customSum = lambda x: (x+x%2)
>>> df4.groupby(level=0).transform(customSum)
```

## Missing Data

```
>>> df.dropna()
>>> df3.fillna(df3.mean())
>>> df2.replace("a", "f")
```

Drop NaN values  
Fill NaN values with a predetermined value  
Replace values with others

## Combining Data

| data1 |        | data2 |        |
|-------|--------|-------|--------|
| X1    | X2     | X1    | X3     |
| a     | 11.432 | a     | 20.784 |
| b     | 1.303  | b     | NaN    |
| c     | 99.906 | d     | 20.784 |

### Merge

```
>>> pd.merge(data1,
 data2,
 how='left',
 on='X1')
```

| X1 | X2     | X3     |
|----|--------|--------|
| a  | 11.432 | 20.784 |
| b  | 1.303  | NaN    |
| c  | 99.906 | NaN    |

```
>>> pd.merge(data1,
 data2,
 how='right',
 on='X1')
```

| X1 | X2     | X3     |
|----|--------|--------|
| a  | 11.432 | 20.784 |
| b  | 1.303  | NaN    |
| d  | NaN    | 20.784 |

```
>>> pd.merge(data1,
 data2,
 how='inner',
 on='X1')
```

| X1 | X2     | X3     |
|----|--------|--------|
| a  | 11.432 | 20.784 |
| b  | 1.303  | NaN    |

```
>>> pd.merge(data1,
 data2,
 how='outer',
 on='X1')
```

| X1 | X2     | X3     |
|----|--------|--------|
| a  | 11.432 | 20.784 |
| b  | 1.303  | NaN    |
| c  | 99.906 | NaN    |
| d  | NaN    | 20.784 |

### Join

```
>>> data1.join(data2, how='right')
```

### Concatenate

#### Vertical

```
>>> s.append(s2)
```

#### Horizontal/Vertical

```
>>> pd.concat([s, s2], axis=1, keys=['one', 'two'])
>>> pd.concat([data1, data2], axis=1, join='inner')
```

## Dates

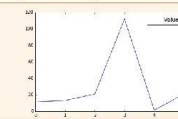
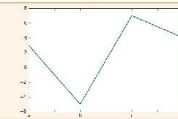
```
>>> df2['Date'] = pd.to_datetime(df2['Date'])
>>> df2['Date'] = pd.date_range('2000-1-1',
 periods=6,
 freq='M')
>>> dates = [datetime(2012, 5, 1), datetime(2012, 5, 2)]
>>> index = pd.DatetimeIndex(dates)
>>> index = pd.date_range(datetime(2012, 2, 1), end, freq='BM')
```

## Visualization

Also see Matplotlib

```
>>> import matplotlib.pyplot as plt
```

```
>>> s.plot()
>>> plt.show()
>>> df2.plot()
>>> plt.show()
```



DataCamp  
Learn Python for Data Science Interactively

